

Not All Perturbations Decontaminate: Classifying Effective vs. Leaky Problem Rewrites in Dynamic Code Benchmarks

Eshgin Hasanov Osama Altarawneh Hongjie Tian
Department of Computer Science, University of Central Florida

eshgin.hasanov@ucf.edu, osama.altarawneh@ucf.edu, ho613779@ucf.edu

1. Problem and Motivation

Task. Code LLMs are benchmarked on public problem sets such as HumanEval [1], which, due to being static in nature and publicly available, often leak into training data and inflate scores. Dynamic benchmarks such as DyCodeEval [2] respond by having an LLM rewrite each problem statement into a new scenario while keeping its semantics and function signature. The assumption is that a model that memorized a solution cannot retrieve it from a rephrased prompt. We ask whether this holds for *each individual rewrite*, and we learn to predict when it does not.

Inputs and outputs. The input is an open-weight model M , an original prompt x_t for task t , and a rewrite $x'_{t,k}$. The output is $p(\text{leaky} \mid x_t, x'_{t,k}, M) \in [0, 1]$. A rewrite is *leaky* if a model contaminated with the canonical solution g_t still reproduces g_t from $x'_{t,k}$, and *effective* if the rewrite breaks this recall.

Why it matters. Our preliminary experiments show that the assumption often fails (Fig. 1). We contaminated StarCoderBase-1B [3] with leaked HumanEval solutions. Under LoRA [4], which trains 0.076% of the parameters, HumanEval rises to 78.7% while DyCodeEval stays at 15.7%, a 63.0 pp gap that makes the rewrites look effective. Under full fine-tuning, the same rewrites break down: DyCodeEval pass@1 rises from 8.7% to 60.9%, and for 96 of 127 tasks at least one rewrite produces the canonical solution *verbatim*, even though the model never saw any rewrite. Whether a perturbation works thus depends on the model and how it was contaminated.

Why it is challenging. Leakiness varies *within* a task. Among the 83 tasks that the contaminated model newly solved, 275 of 415 rewrites (66%) recalled the solution verbatim, and 46 tasks had a mix of leaky and effective rewrites. Simple similarity also fails. In our earlier analysis, raw cosine similarity between prompt embeddings separated matched from unrelated prompts with AUROC of only

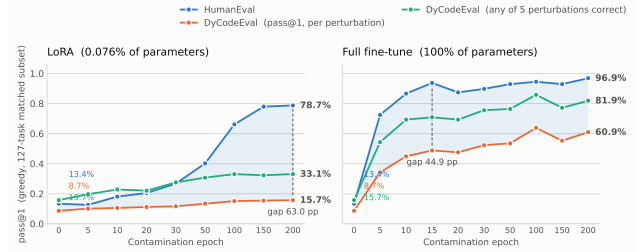


Figure 1. Motivating example: StarCoderBase-1B, 100% leakage (greedy pass@1, 127 tasks). Under LoRA, DyCodeEval stays flat and the rewrites seem to work. Under full fine-tuning, it climbs to 60.9%, so many rewrites no longer block memorization.

0.54, because decoder embeddings are highly anisotropic (matched and unrelated pairs differ by only 0.0035 in cosine).

Research question. *Can features from a model’s prompt representations, and from how fine-tuning changes them, predict per rewrite whether memorized-solution recall survives the perturbation, and can they do so better than text-only baselines, reaching AUROC ≥ 0.75 on held-out tasks?*

2. Related Work and Research Gap

Dynamic benchmarks. DyCodeEval [2] rewrites HumanEval/MBPP problems into new scenarios with an LLM, and EvoEval [5] evolves problems into new variants. Both validate their rewrites *on average* and treat every variant as an equally valid probe. **Contamination detection.** Likelihood-based detectors such as Min-K% Prob [6] decide whether a *model* saw a given text. **Gap.** No existing method evaluates whether an *individual rewrite* still acts as a retrieval key for a memorized answer, and none predicts this per model. Our project fills this gap.

3. Proposed Method

Goal. A binary classifier $f(x_t, x'_{t,k}; M) \in \{\text{good}, \text{bad}\}$ that tells whether a generated perturbation is *good*, i.e.,

whether it stops a contaminated model from reproducing the memorized solution.

- **Labels.** We contaminate M by fine-tuning it on leaked solutions and generate code from each rewrite. A rewrite is *bad* if the generated function body matches the canonical solution g_t (edit similarity ≥ 0.95) and *good* otherwise.
- **Features.** We embed x_t and $x'_{t,k}$ with M (mean-pooled hidden states) and use (i) their cosine similarity at each layer, (ii) how much fine-tuning moves each embedding, and (iii) simple text overlap between the two prompts.
- **Classifier.** Logistic regression and a small MLP over these features.
- **Models.** StarCoderBase [3] (1B, 15.5B), contaminated with full fine-tuning and with LoRA [4]. MPNet and Qwen3-Embedding serve as text-only baselines.
- **Course topics:** representation learning, fine-tuning of pretrained models, supervised classification, and cross-validation.

4. Dataset and Experiments

Data. HumanEval [1] has 164 Python problems with canonical solutions and tests. DyCodeEval [2] provides 5 rewrites for each of 127 of these problems, 635 in total. Our contaminated checkpoints (already trained) are StarCoderBase-1B with full fine-tuning and LoRA, and 15.5B with LoRA, each at 25/50/75/100% leakage and up to 10 epoch milestones. This gives on the order of 10^4 labeled triples. Unleaked tasks at partial leakage serve as negative controls. **Splits.** We use 5-fold GroupKFold *by task*, so that rewrites of one problem never appear in both train and test. **Metrics.** We report AUROC (primary), AUPRC, F1, and calibration error. **Baselines.** The baselines are random guessing, a threshold on raw last-layer cosine, surface features only, and MPNet/Qwen3 text-encoder features. **Principal comparisons.** (E1) Analysis: how each feature relates to whether a rewrite is good or bad. (E2) Classification: our classifier vs. the baselines. (E3) Generalization: train on one condition and test on another, across epochs, leakage levels, full→LoRA, and 1B→15.5B. (E4) Benchmark repair: keep only rewrites predicted to be good, and check that contamination inflation, $\text{pass@1}(\text{contaminated}) - \text{pass@1}(\text{base})$, shrinks more than under random filtering. **Ablations.** We remove each feature group (i)–(iii) and vary the 0.95 similarity threshold.

5. Novelty

Prior work *generates* perturbed problems [2, 5] or detects contamination at the *model* level [6]. This project

goes beyond reproducing either. It (i) shows under controlled, known contamination that many semantics-preserving rewrites still retrieve memorized solutions verbatim, with this varying across rewrites of the same problem and across fine-tuning regimes; (ii) learns the first *per-rewrite*, *per-model* classifier of perturbation effectiveness from representation geometry and its fine-tuning shift; and (iii) uses the classifier to filter a dynamic benchmark, turning “perturb and hope” into “perturb and verify.”

6. Feasibility and Fallback Plan

Data and compute. All datasets are public, and all contaminated checkpoints, generations, and pass/fail results already exist. The remaining compute is feature extraction: forward passes over about 800 prompts per checkpoint, which takes minutes per 1B checkpoint on one GPU of the university cluster. The classifiers train on a CPU. **Risks and fallbacks.** (1) If embedding features are weak, we fall back to text-overlap features and report the negative result. (2) If the labels are too imbalanced (e.g., under LoRA), we lower the threshold or predict the edit similarity directly. (3) If transfer across models fails, we train a per-model classifier, which is the stated goal. (4) With only 127 independent tasks, overfitting is a risk, so we use regularized models, grouped splits, and bootstrap confidence intervals.

7. Timeline and Responsibilities

- **Wk 1–2:** per-layer feature extraction for all checkpoints; build the label table.
- **Wk 3–4:** E1 correlation analysis; baselines.
- **Wk 5–6:** E2 classifiers and ablations; *milestone report*.
- **Wk 7–8:** E3 generalization; E4 benchmark repair.
- **Wk 9–10:** final analysis, figures, and *final report*.

Responsibilities. *Eshgin Hasanov:* contaminated checkpoints, feature extraction, and labeling. *Osama Altarawneh:* classifiers, baselines, and ablations (E1, E2). *Hongjie Tian:* generalization and benchmark-repair experiments (E3, E4). All members share the writing of the milestone and final reports.

References

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021. 1, 2
- [2] S. Chen, P. Pularla, and B. Ray, “DyCodeEval: Dynamic benchmarking of reasoning capabilities in code large language models under data contamination,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2025. 1, 2

- [3] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, *et al.*, “Star-Coder: May the source be with you!,” *Transactions on Machine Learning Research*, 2023. [1](#), [2](#)
- [4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022. [1](#), [2](#)
- [5] C. S. Xia, Y. Deng, and L. Zhang, “Top leaderboard ranking = top coding proficiency, always? EvoEval: Evolving coding benchmarks via LLM,” in *Conference on Language Modeling (COLM)*, 2024. [1](#), [2](#)
- [6] W. Shi, A. Ajith, M. Xia, Y. Huang, D. Liu, T. Blevins, D. Chen, and L. Zettlemoyer, “Detecting pretraining data from large language models,” in *International Conference on Learning Representations (ICLR)*, 2024. [1](#), [2](#)